

BUILD SPECIFICATION · V1.0

IMan Afrikah World

The complete developer specification for building the first Artist World on songchainn — architecture, features, data model, token gating, and the phased build path.

ARTIST WORLDS BY SONGCHAINN · WORLD #001

PREPARED FOR

The songchainn developer

ACCESS TOKEN

\$IMAN · Base · Clanker v4

STACK

Next.js 15 · React 19 · TypeScript ·
Tailwind v4

PREPARED BY

IMan Afrikah's office

SURFACE

songchainn.xyz · Farcaster Mini App
(FID 1057869)

DATE

18 July 2026

Companion to IMAN_WORLD_CONCEPT.md · IMAN_CLANKER_LAUNCH_PLAYBOOK.md · IMAN_NEXT_MOVE_2026-07-15.md. Internal build document. Copy rule in force everywhere: **sell access and belonging, never price.**

00 What this document is

This is the build brief the songchainn developer works from. It turns the World concept into a concrete engineering plan: what to build, in what order, on which parts of the existing stack, and exactly how token gating, content, and status are wired. Everything here is grounded in what already exists — an 86-track onchain catalog, the Zora song coins, the \$IMAN Clanker deploy, and the live Next.js mini app.

THE ONE-SENTENCE THESIS

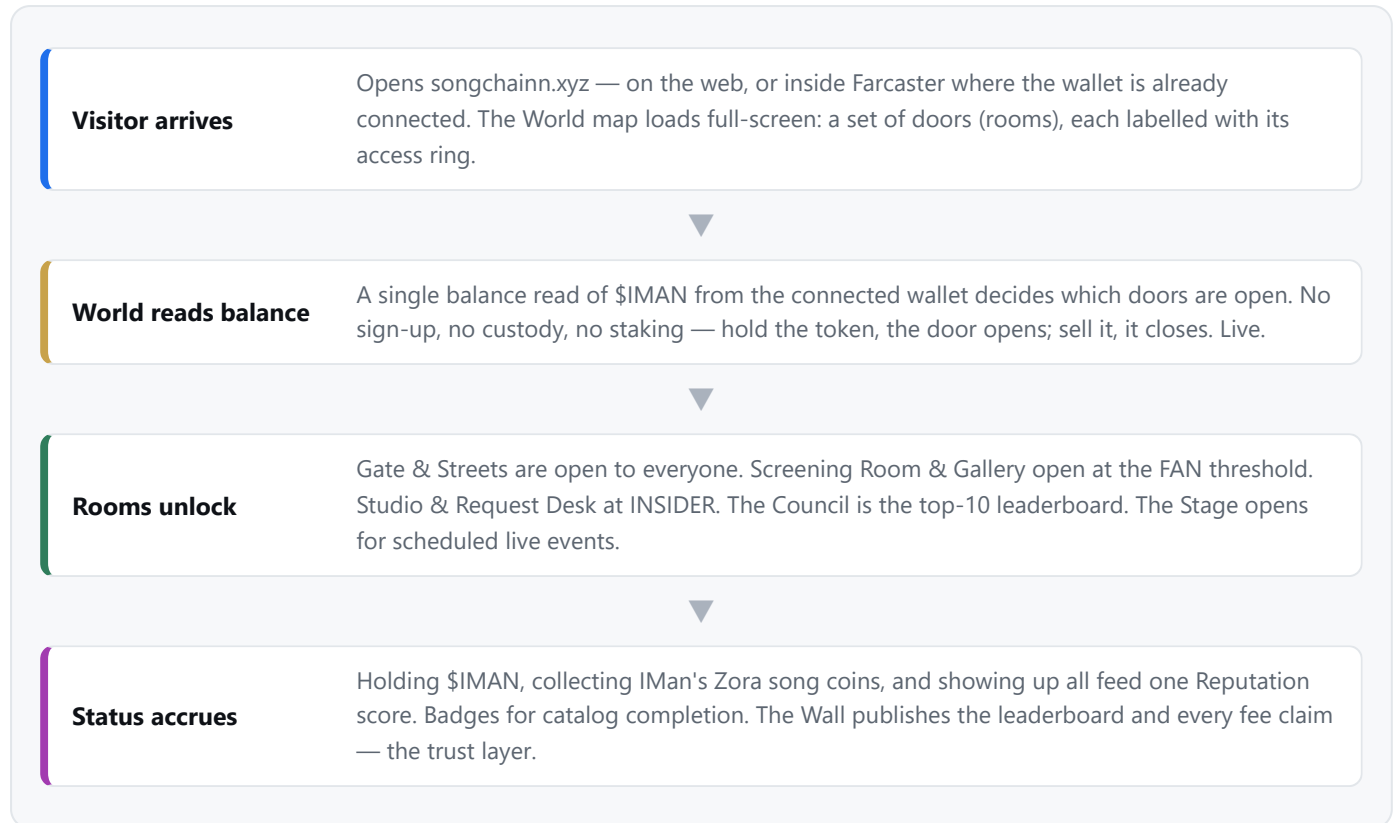
An artist profile should be a **world**, not a page. We prove the thesis by building the first one ourselves. Rooms are access rings gated by how much \$IMAN a wallet holds; every room fills with content IMan already produces; one status score ties holding, collecting, and showing up into a single game. Ship it as a gated hub first, add live moments, and let it graduate to a spatial 3D world when songchainn launches that vision.

Contents

- 01 The product in one picture — what a citizen experiences
- 02 The world, room by room — access rings & content
- 03 System architecture — how it plugs into songchainn
- 04 The gating engine — reading \$IMAN & enforcing rings
- 05 Data model — content, rooms, citizens, reputation
- 06 Feature-by-feature build spec
- 07 The Reputation engine — two engines, one score
- 08 Token & economy wiring — Splits, vault, fees
- 09 Build path — Phase A / B / C with milestones
- 10 Tech stack, repo layout & new dependencies
- 11 Security, trust & the copy rule
- 12 Definition of done & developer checklist

01 The product in one picture

A world is a full-screen immersive layer that lives on top of the existing songchainn site. A visitor lands at The Gate, connects a wallet, and the world reads their \$IMAN balance. Doors they can open light up; doors they can't show what it takes to enter. Everything behind a door is content IMan controls.



What's genuinely new to build

The World shell (map/doors UI), the wallet + balance gating engine, the room content system, the Reputation/leaderboard service, the Request Desk, and the Stage (live events). All layered onto the current app.

What already exists and gets reused

The 86-track onchain catalog, the deployed Zora song coins, the Next.js 15 / React 19 site, the Farcaster Mini App registration & wallet context, the content data modules, and \$IMAN itself.

DESIGN PRINCIPLE FOR THE DEVELOPER

The world is a **layer, not a rewrite**. Everything is additive to the existing app. Ring 0 rooms (Gate, Streets) are essentially the current homepage restyled into the world frame, so day one is not blocked on any gated feature being finished.

02 The world, room by room

Rooms are access rings. The deeper into the world, the more \$IMAN a wallet holds. Balance is checked live from the connected wallet. These names are the concept-locked set — treat them as fixed strings in the codebase (room slugs) so Phase C can keep the same rooms while adding geometry.

RING	ROOM	WHO GETS IN	WHAT'S INSIDE
0	The Gate	Everyone, no wallet	World trailer, 3 featured tracks (previews), the story, "Get \$IMAN" swap widget, join links
0	The Streets	Everyone	Public feed: latest drops from the 86-track onchain catalog, hot-today chart, collect buttons (Zora song coins)
1	The Screening Room	Hold ≥ FAN	Music videos, interviews, documentaries, visualizers. Premieres land here first
1	The Gallery	Hold ≥ FAN	Photos, cover-art archive, lyric sheets, collectible image drops
2	The Studio	Hold ≥ INSIDER	Works-in-progress, demos, stems, voice notes, beat previews from IManOnTheBeat, occasional live session streams
2	The Request Desk	Hold ≥ INSIDER	Submit song requests; monthly voting; see the request queue
3	The Council	Top-10 leaderboard	Guaranteed monthly picks, named in release credits, private chat with IMan, first listen before any drop
E	The Stage	Event-based (any holder)	Scheduled listening parties, premiere nights, Q&A. Council gets front row (early-access link)
W	The Wall	Public, read-only	Supporters wall: leaderboard, credits history, transparency reports (fees claimed, treasury spend)

THRESHOLDS ARE CONFIG, NOT CODE

FAN and INSIDER thresholds are stored as editable config (in \$IMAN units), reviewed monthly as price discovers. Target at entry: FAN ≈ \$5–10 equivalent, INSIDER ≈ \$50–100 equivalent. The developer must never hard-code a threshold; it reads from a single config source so IMan + Zaal can tune it without a deploy.

RING 0 SHIPS FIRST, UNBLOCKED

The Gate and The Streets need no gating logic — they are public. They can go live the day \$IMAN launches, essentially reframing the current homepage, while the gated rooms are built behind them.

Room states the UI must render

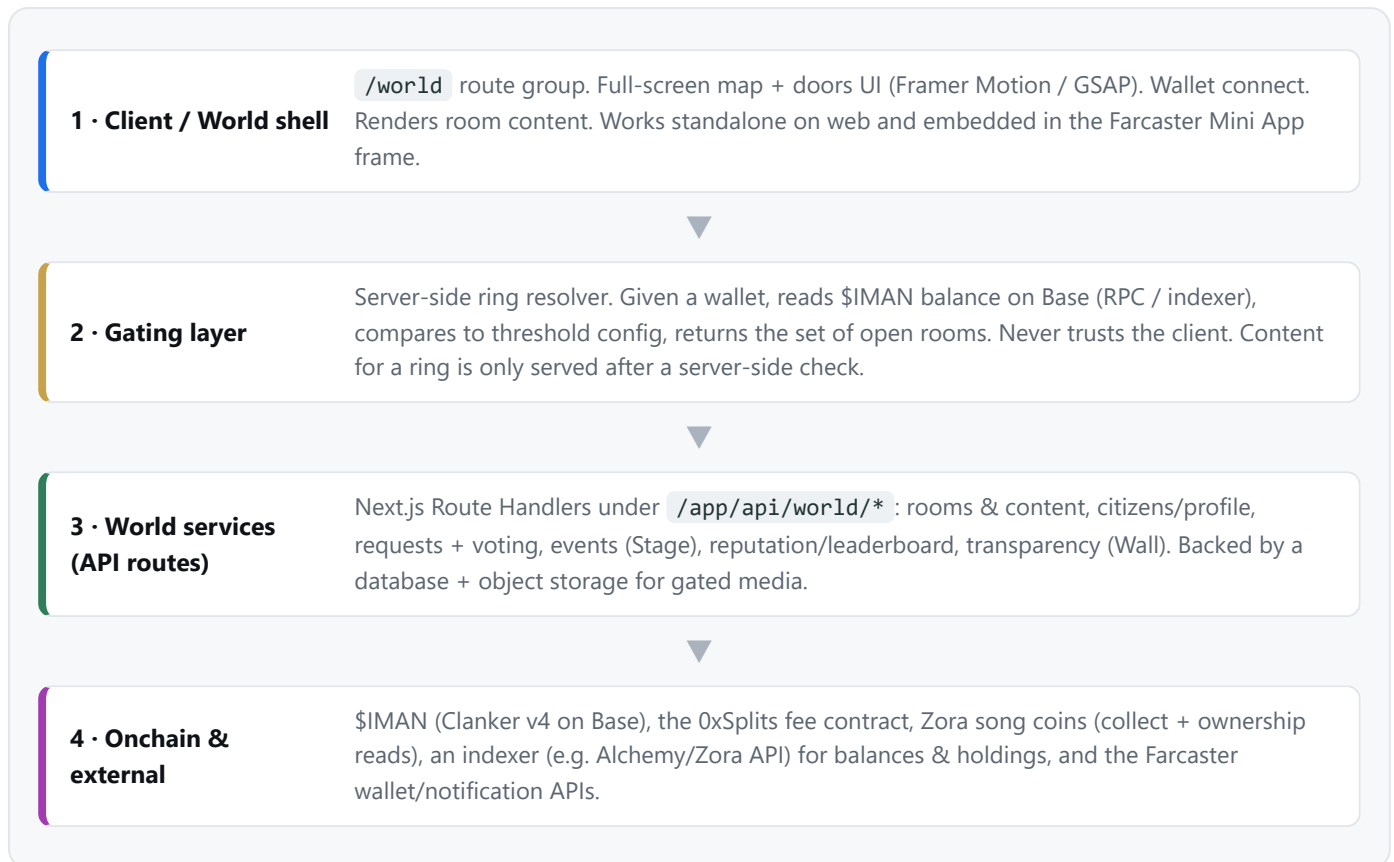
STATE	WHEN	WHAT THE DOOR SHOWS
Open	Wallet meets the ring	Room is enterable; door lit; content loads inside
Locked	Wallet below threshold	Door dimmed; overlay: "Hold ≥ X \$IMAN to enter" + inline swap CTA; a teaser (blurred cover / count of items inside)
No wallet	Not connected	"Connect to see your doors" + connect button; Ring 0 still fully usable

STATE	WHEN	WHAT THE DOOR SHOWS
Event live	Stage event running now	The Stage pulses "LIVE"; deep-link into the stream/chat
Council	Wallet in top-10	Council door shows a crown; private features enabled

03 System architecture

The World is a new route group and a set of services inside the existing `webapp` (Next.js 15 App Router). It reuses the app's rendering, content modules, and Farcaster context, and adds three things the current site doesn't have: a wallet/balance read, a gating layer, and a lightweight backend for citizens, requests, events, and reputation.

The four layers



Request flow — opening a gated room

- 1 Visitor opens a room.** Client calls `GET /api/world/room/[slug]` with the connected wallet address (and, in-frame, the Farcaster-verified address).
- 2 Server resolves the ring.** Gating layer reads the wallet's \$IMAN balance from the indexer, compares against the room's required ring threshold from config.
- 3 Decision.** If below threshold → return a *locked* payload (teaser + required amount, no media URLs). If met → proceed.
- 4 Serve gated content.** For open rooms, the server returns content records and issues **short-lived signed URLs** for the actual media so gated files can't be hotlinked or shared past expiry.
- 5 Client renders.** World shell paints the room; balance is re-checked periodically and on wallet change so a door closes if the holder sells.

NON-NEGOTIABLE: GATE ON THE SERVER

Balance checks and content access decisions happen **server-side**. The client may optimistically dim/undim doors for responsiveness, but no gated media URL is ever sent to a wallet that hasn't been verified to meet the ring on the server. Client-only gating is trivially bypassed and must not be the enforcement boundary.

04 The gating engine

This is the heart of the world and the one piece with real security weight. It answers a single question fast and correctly: *for this wallet, which rooms are open right now?*

Inputs

- **Wallet address** — from web wallet-connect, or the Farcaster-verified address when running in-frame (preferred, since it's attested by the client).
- **\$IMAN balance** — read from an indexer/RPC on Base for the \$IMAN contract. Cached briefly (see below).
- **Threshold config** — `{ FAN: n, INSIDER: n }` in \$IMAN units, from a single editable config source.
- **Leaderboard position** — for Council (ring 3), resolved from the reputation service, not from balance alone.

The resolver (shape, not final code)

```
// app/api/world/_lib/gate.ts - server only
export async function resolveRings(wallet) {
  const bal    = await getImanBalance(wallet); // indexer read, cached ~30s
  const cfg    = await getThresholds();        // { FAN, INSIDER }
  const rank   = await getLeaderboardRank(wallet); // reputation service

  return {
    ring0: true, // Gate + Streets: always
    ring1: bal >= cfg.FAN, // Screening Room + Gallery
    ring2: bal >= cfg.INSIDER, // Studio + Request Desk
    council: rank !== null && rank <= 10, // The Council
    balance: bal, thresholds: cfg, rank
  };
}
```

Every room maps to a required ring

ROOM SLUG	REQUIRES	CHECK
gate , streets , wall	Ring 0	Always open
screening-room , gallery	Ring 1 (FAN)	balance ≥ FAN
studio , request-desk	Ring 2 (INSIDER)	balance ≥ INSIDER
council	Ring 3	leaderboardRank ≤ 10
stage	Event	Any holder during a scheduled event; Council early-access link

CACHING & FRESHNESS

Cache balance reads ~30s per wallet to avoid hammering the RPC on every navigation, but re-resolve on wallet change and on explicit "refresh my access." A door should close within a minute of a holder selling below threshold — near-live, not instant, is the right trade.

NON-CUSTODIAL BY DESIGN

No staking, no locking, no deposits to start. The world only ever *reads* balance. This keeps it simple, safe, and legally light — holders keep full custody, and "hold to enter" is the entire mechanic.

ANTI-ABUSE

Rate-limit balance/gate endpoints per IP + wallet. For Council/private features, verify wallet ownership (Farcaster attestation in-frame, or a sign-in-with-Ethereum signature on web) so a rank can't be spoofed by passing someone else's address.

05 Data model

The world needs a small backend. Content that is public and static (Ring 0 catalog, the story) can stay in the existing typed data modules; anything gated, dynamic, or per-citizen (requests, votes, events, reputation, signed media) needs a database. Below is the minimum schema.

Core entities

ENTITY	KEY FIELDS	NOTES
Room	slug, name, requiredRing, order, doorArt	Seeded from the fixed room set; rarely changes
ContentItem	id, roomSlug, type (video/photo/audio/doc/wip), title, mediaKey, premiereAt, isPremiere, catalogTag	mediaKey → object storage; served via signed URL only after gate passes
Citizen	wallet, fid, handle, joinedAt, reputation, badges[], streak	One row per wallet that has entered; profile in the world
SongCoinHolding	wallet, coinId, catalogTag, collectedAt	Synced from Zora/indexer; drives catalog-completion badges
Request	id, wallet, text, season, status, votes, createdAt	Ring 2 gated to submit; queue is visible
Vote	requestId, wallet, season	Monthly cycle; one weight rule per citizen
Event (Stage)	id, title, startsAt, streamUrl, type, councilEarlyLink	Drives the Stage LIVE state + notifications
ReputationEvent	wallet, kind, points, ts	Append-only ledger; leaderboard is a sum/rollup
FeeClaim	id, amount, txHash, claimedAt, splitBreakdown	Published on The Wall for transparency

Storage split

DB (Postgres via a hosted provider) for the entities above.
Object storage (S3-compatible / R2) for gated media, always fronted by short-lived signed URLs. **Public static** data stays in the existing `src/content/data` modules.

Onchain is the source of truth for money

Balances, holdings, and fee claims are read from chain / indexer — the DB only *caches and annotates* them (badges, tags, reputation). Never treat the DB as authoritative for who holds what.

WHY A REAL BACKEND NOW

The current site is static/typed-data only. Requests, votes, events, per-citizen reputation, and signed gated media all need writes and per-wallet state. Keep it lean — a single Postgres instance and one object-storage bucket cover Phase A and B comfortably.

06 Feature-by-feature build spec

Each room/feature, with what the developer builds, the endpoints, and the reused vs. new parts.

The Gate & The Streets Ring 0 ships first

- Full-screen world entry: trailer, story, 3 featured track previews, "Get \$IMAN" swap widget, join links.
- The Streets = the public feed of the 86-track onchain catalog with a hot-today chart and Zora collect buttons.
- **Reuse:** existing discography data module, existing home components. **New:** the world frame/shell, swap widget embed, collect button wiring to Zora.
- **Endpoints:** mostly static + `GET /api/world/streets` (feed, hot chart).

The Screening Room & The Gallery Ring 1 · FAN

- Gated media grids: videos/interviews/visualizers (Screening Room); photos/cover art/lyric sheets (Gallery). Premieres land here first.
- **New:** gated media pipeline (signed URLs), premiere scheduling, lightbox/player reused from the site's gallery components.
- **Endpoints:** `GET /api/world/room/screening-room`, `.../gallery` — return locked payload or content + signed URLs.

The Studio & The Request Desk Ring 2 · INSIDER

- Studio: WIPs, demos, stems, voice notes, beat previews, occasional live session embeds.
- Request Desk: submit a request, monthly voting, visible queue. This is the engine of "fans funding the music."
- **New:** request submit + queue + monthly voting cycle; upload/admin for IMan to post Studio items.
- **Endpoints:** `POST /api/world/requests`, `GET /api/world/requests?season=`, `POST /api/world/requests/[id]/vote`.

The Council Ring 3 · top-10

- Top-10 leaderboard members: guaranteed monthly picks, credit in releases, private chat with IMan, first listen.
- **New:** rank-gated access (not balance-gated), private chat surface (can start as a gated channel link), first-listen drop mechanism.
- **Endpoints:** `GET /api/world/council` (verifies rank ≤ 10 with wallet ownership proof).

The Stage Event

- Scheduled listening parties / premiere nights / Q&A: embedded stream or audio + live chat; Council gets a front-row early link.
- **New:** event scheduler, LIVE state on the world map, push notifications to holders via the Mini App, embedded stream + chat.
- **Endpoints:** `GET /api/world/events`, `GET /api/world/events/[id]`; notification trigger on event start.

The Wall Public

- Read-only trust surface: leaderboard, credits history, transparency reports (fees claimed, treasury spend, the Splits address).
- **New:** render `FeeClaim` records + leaderboard publicly; link the onchain Splits contract.
- **Endpoints:** `GET /api/world/wall` (leaderboard + fee-claim history).

07 The Reputation engine — two engines, one score

World rank ("Reputation") is the single status number that makes the whole thing a game instead of a paywall. Two fee engines and one activity stream all feed it, so whales alone don't own the culture.

1 · Holding \$IMAN

Opens rooms directly. Clanker LP fees flow on every trade. Balance is the ring key.

2 · Collecting song coins

Collecting IMan's Zora content coins earns Reputation + badges per catalog collected. Zora creator rewards flow on those trades.

3 · Activity

Attending Stage events, fulfilled requests, streaks → non-transferable points. Can't be bought.

Scoring model (tunable)

```
// reputation = weighted sum, recomputed from the ReputationEvent ledger
reputation(wallet) =
  w1 * imanBalanceScore(wallet)      // holding
+ w2 * songCoinsCollected(wallet)   // collecting (per-catalog)
+ w3 * activityPoints(wallet)        // events, requests, streaks (non-transferable)

// leaderboard = citizens ordered by reputation desc; top 10 = Council
```

Badges (catalog completion)

Own every track in a catalog to earn that catalog's badge — e.g. collect all **HIGH-BRED** tracks → the HIGH-BRED badge. Badges add Reputation and are shown on the citizen profile and The Wall.

HIGH-BRED — complete

FARCASTER SONG — collected

Founding Citizen

Council (current season)

Request Fulfilled

Stage Regular — 5 events

WHY THE ACTIVITY LEG MATTERS

Pure balance ranking would make the world a whale board. The non-transferable activity points mean showing up, collecting the catalog, and getting requests made all move you up — that's what keeps the culture alive and the trading volume (the thing that actually pays fees) flowing.

IMPLEMENTATION NOTE

Keep `ReputationEvent` append-only and recompute rollups on a schedule (and on demand for a single wallet). Weights `w1/w2/w3` live in the same tunable config as thresholds so the economy can be balanced without redeploying.

08 Token & economy wiring

The world reads the token; it does not reimplement the token. The economics were decided in the launch playbook and are locked. The developer's job is to read balances, surface the Splits/vault transparently, and never introduce custody.

PIECE	WHAT IT IS	WORLD'S JOB
\$IMAN	Clanker v4 token on Base, 1% fee tier	Read balance for gating; show "Get \$IMAN" swap widget
0xSplits contract	Sole fee recipient; 50 IMan / 30 treasury / 20 studio, mutable inside Splits	Publish the Splits address on The Wall; render the current split
Vault	20% of supply, 90-day lock, linear vesting	Fund airdrops to active citizens, Request Season prizes, event rewards; show reserve status
Zora song coins	Per-track collectibles already deployed	Collect buttons in The Streets; drive badges + Reputation via holdings sync
Fee claims	Creator-side rewards claimed at clanker.world	Record each claim + tx hash → publish on The Wall

THE WALL = THE TRUST ENGINE

Every fee claim and the live Splits breakdown are published publicly. This is the single most important trust signal — it's what makes "back the album" credible without ever promising a return. Build The Wall to render straight from onchain data + the FeeClaim ledger.

IMMUTABLE FEE SPLIT — ALREADY HANDLED

Clanker v4 `rewardBps` are immutable forever. That's why the sole recipient is a mutable 0xSplits contract. The developer must **never** wire gating or payouts assuming the split can change onchain — only the Splits internal shares change, and only by its controller.

VAULT-FUNDED REWARDS, PHRASED CAREFULLY

Airdrops and Season prizes are funded from the vault, but copy presents them as what citizens *currently* enjoy — never as guaranteed entitlements or promised payouts (see §11). The developer surfaces vault activity as transparency, not as a promise.

09 Build path — Phase A / B / C

Ship the world as a gated hub first (real product, not vaporware), add live moments, then let it graduate to a spatial world when songchainn launches that vision. The rooms, gates, and content survive across all three phases — only the presentation gains dimension.

Phase A The World as a gated hub · 2–3 weeks

A distinct immersive layer on songchainn.xyz: full-screen world map / doors UI, each room a gated section. Wallet connect + \$IMAN balance check per ring. Works inside the existing Farcaster Mini App (wallet already connected, notifications for premieres). This ships fast and *is* the world in product terms.

- World shell + doors UI; Ring 0 (Gate + Streets) live day one.
- Gating engine (balance read + threshold config + server enforcement).
- Screening Room + Gallery (FAN); Studio + Request Desk (INSIDER); Council (top-10).
- Backend: rooms, content, citizens, requests, reputation/leaderboard, signed media.
- The Wall v1: leaderboard + first transparency report.

Phase B Live moments · weeks 4–8

The Stage: scheduled premieres and listening parties (embedded stream/audio + live chat), push notifications to holders via the Mini App. Request Desk voting cycles begin (Request Season 1).

- Event scheduler + Stage LIVE state + notifications.
- Monthly request/voting cycle; Council first-listen + private chat.
- Badges + catalog-completion sync fully live.

Phase C The spatial world · when songchainn launches the vision

The hub graduates to a navigable 3D space (three.js / react-three-fiber on web first; VR headsets later). Rooms keep the same names, gates, and content — the world was already alive, it just gains geometry. IMan's world becomes the template other artists clone with their own access token.

- 3D navigable space over the same room/gate/content model (no data migration — the API is unchanged).
- "Artist Worlds by songchainn" productized: clone the template, swap the token, launch world #002.

MILESTONE	PHASE	GATE TO MOVE ON
Ring 0 live on token launch day	A	Gate + Streets reachable in-frame and on web
Gated rooms enterable	A	Server-side gating verified; no media leaks to sub-threshold wallets
Request Season 1 opens	A→B	Request submit + queue live at INSIDER
First Stage event	B	Scheduler + notifications + embedded stream working
Grant application	B	"Artist-controlled, token-gated world, live as a Mini App" demonstrable
Spatial prototype	C	songchainn commits to the 3D vision

10 Tech stack, repo layout & new dependencies

Build inside the existing `webapp`. Keep the current stack; add only what gating, backend, and onchain reads require.

ALREADY IN THE REPO (REUSE)

Next.js 15 (App Router) React 19 TypeScript
Tailwind v4 Framer Motion GSAP
shadcn / @base-ui lucide-react Zod Resend
MDX content

Route Handlers under `/app/api` already exist (the contact form). The gallery lightbox, content-list, and home components are directly reusable inside rooms.

NEW DEPENDENCIES TO ADD

viem / wagmi Farcaster Mini App SDK
Zora coins SDK / API Indexer (Alchemy / Zora API)
Postgres (hosted) ORM (Drizzle / Prisma)
Object storage (R2 / S3) SIWE (web wallet auth)

Keep it minimal: onchain reads (viem + an indexer), a DB with an ORM, object storage with signed URLs, and the Farcaster SDK for in-frame wallet + notifications.

Proposed repo layout (additive)

```
webapp/src/
├─ app/
│  ├─ world/                    # the World route group (full-screen shell)
│  │  ├─ layout.tsx            # world frame, wallet provider, map chrome
│  │  ├─ page.tsx              # the map / doors (room states)
│  │  └─ [room]/page.tsx       # a single room (gated render)
│  └─ api/world/
│     ├─ room/[slug]/route.ts  # gated content + signed URLs
│     ├─ streets/route.ts      # public catalog feed + hot chart
│     ├─ requests/route.ts     # submit / list / vote
│     ├─ events/route.ts       # Stage schedule + live state
│     ├─ council/route.ts      # rank-gated (ownership proof)
│     ├─ reputation/route.ts   # leaderboard + score
│     └─ wall/route.ts         # fee claims + transparency
├─ world/                      # world domain logic (non-route)
│  ├─ gate.ts                  # resolveRings() – server only
│  ├─ balance.ts               # indexer reads + cache
│  ├─ reputation.ts            # scoring + badges
│  ├─ media.ts                 # signed URL issuance
│  └─ config.ts                # thresholds + weights (editable)
├─ db/                         # schema + migrations (ORM)
└─ components/world/          # doors, room shells, badges, Wall
```

FRAMEWORK NOTE (FROM THE REPO)

This project runs on a **customized Next.js build with some non-standard APIs** — check `node_modules/next/dist/docs/` before relying on upstream Next.js docs for framework-level changes (see `AGENTS.md`). Factor this into any App Router / Route Handler work.

11 Security, trust & the copy rule

Security must-dos

AREA	REQUIREMENT
Gate enforcement	Server-side only. No gated media URL reaches a sub-threshold wallet. Client gating is cosmetic.
Media protection	Short-lived signed URLs for all gated files. No public bucket paths. Expire & rotate.
Wallet ownership	Council/private features require proof of ownership (Farcaster attestation in-frame or SIWE on web) — never trust a passed-in address for privileged actions.
Rate limiting	Throttle gate/balance/request endpoints per IP + wallet. Cache balance reads to protect the RPC.
No custody	The world never holds funds or tokens. Read-only onchain. Zero deposit/stake/lock in Phase A–B.
Secrets	Indexer keys, DB creds, storage creds server-side only (env vars); never shipped to the client.
Input validation	Zod-validate every write (requests, votes). Prevent vote/request spam per citizen per season.

THE COPY RULE — APPLIES TO EVERY STRING THE DEVELOPER SHIPS

Sell access and belonging, never price. No profit talk, ever. Present perks as what holders *currently enjoy* ("holders currently get..."), never as guaranteed entitlements ("holders will receive..."). No promised rewards, treasury payouts, or future benefits in UI copy. Positioning line: "\$IMAN is not a fan token. It is the key to my world." Frame everything as "*back the album*," not "buy a coin."

UI COPY HOUSE RULE (THIS MACHINE)

Never use the em dash character in any user-facing UI text (website/app copy, component text). Rephrase with a comma, period, or colon. This applies to every string rendered inside the world. (Em dashes are fine in this doc, commit messages, and code comments — only visible product copy is restricted.)

LEGAL-LIGHT UNTIL COUNSEL REPORTS

All public-facing world copy stays light on promises until Zaal's counsel reports back. When in doubt, describe the present, not the future; describe access, not returns.

12 Definition of done & developer checklist

Phase A is "done" when a wallet holding the FAN threshold can walk into the Screening Room and Gallery, a wallet holding INSIDER can reach the Studio and submit a request, the top-10 sees the Council, and no sub-threshold wallet can pull a single gated media file. Below is the ordered checklist.

Phase A checklist

#	TASK	LAYER
1	Scaffold <code>/world</code> route group + full-screen shell + doors map with room states (open / locked / no-wallet)	Client
2	Wallet connect: Farcaster in-frame address + web wallet-connect; SIWE for privileged actions	Client / Auth
3	Balance read on Base via indexer + short cache; threshold config source	Gating
4	<code>resolveRings()</code> server resolver; wire every room to a required ring	Gating
5	DB schema + ORM + migrations for the core entities	Server
6	Object storage + signed-URL media pipeline (gated files only)	Server
7	Ring 0: Gate + Streets (reuse discography/home; add swap widget + Zora collect)	Feature
8	Ring 1: Screening Room + Gallery (gated media grids, premieres)	Feature
9	Ring 2: Studio (WIP feed) + Request Desk (submit + queue)	Feature
10	Reputation service + leaderboard; Council rank-gating (top-10)	Feature
11	The Wall v1: leaderboard + first transparency report + Splits address	Feature
12	Security pass: server-gate audit, signed-URL expiry, rate limits, ownership proof	Hardening
13	Copy pass: access-not-price, no em dashes, legal-light	Copy

The five decisions — recommended defaults & when to decide

Key finding: **none of these hard-block Phase A.** The architecture makes thresholds editable config and the room names are already concept-locked, so the build starts on two locked decisions and the other three follow within the first week with sensible placeholders in the meantime.

#	DECISION	RECOMMENDED DEFAULT	WHEN
2	Room names code slugs	Lock the concept set as-is: <code>gate</code> · <code>streets</code> · <code>screening-room</code> · <code>gallery</code> · <code>studio</code> · <code>request-desk</code> · <code>council</code> · <code>stage</code> · <code>wall</code> . Already decided in the World doc — builder codes them verbatim. Changing a slug later = a migration, so confirm now (free) rather than revisit.	NOW structural — needed at handoff
5	Copy rule every string	Confirm it stands: <i>access and belonging, never price</i> ; no profit talk; "holders currently enjoy," never "will receive;" no em dashes in UI text; legal-light until counsel reports. It's already your standing rule — this just rides along in the handoff as the guardrail for all copy.	NOW guardrail — confirm at handoff
1	FAN / INSIDER thresholds config, not code	Ship with placeholders; tune after price discovers — the whole point of config. Launch deliberately low to grow the citizen base : FAN = a small round token amount (any genuine holder gets in), INSIDER $\approx 10\times$ FAN. Re-peg monthly to	WEEK 1 send after launch

#	DECISION	RECOMMENDED DEFAULT	WHEN
		the dollar targets (FAN ≈ \$5–10, INSIDER ≈ \$50–100 equivalent) once \$IMAN has a real price.	
3	Branding surface only	Lean: " Artist Worlds by songchainn — World #001: IMan Afrikah " from day one . Costs nothing extra to build, plants the platform story early (the grant angle + "IMan gets paid twice" upside), and can always be de-emphasized. Retrofitting the platform framing later is harder than leading with it. Touches copy/logo lockup, not architecture.	WEEK 1 can follow
4	Content cadence ops, not arch	The builder builds the same room system regardless; cadence only sets how much admin/upload tooling to prioritize. Suggested default stands: 1 Streets drop/day from catalog · 1 Studio item/week · 1 Screening Room premiere/month · 1 Stage event/month. Confirm during Phase A once the real workload is felt.	WEEK 1 confirm in Phase A

CLEAN-HANDOFF INSTRUCTION

Send the builder **#2 and #5 locked** (room slugs + copy rule) plus a note that **#1, #3, #4 follow within the first week** with the placeholders above in the meantime. This avoids stalling the build on decisions IMan + Zaal are better off making once the token is live and price is real.

THE PAYOFF FOR GETTING THIS RIGHT

Every piece of content IMan already makes becomes gated inventory instead of a free social post — zero new production cost, a concrete reason to hold, and the first artist world on Farcaster. When it works, "launch your world" becomes songchainn's product, and IMan gets paid twice: as the artist of his world, and as the platform behind everyone's.